Learning for Robotics

Week 1: Foundations

By Fatemeh Hafezianzadeh

What Is a Robot?

A machine that can SENSE, DECIDE, and ACT

Factory Arm

Self-Driving Car

Drone

Robot Vacuum

Surgical Robot

Simulated Agent

The Robotics Pipeline

Every robot needs these three abilities

1 Perception

Understand the world from sensors & cameras

2 Planning

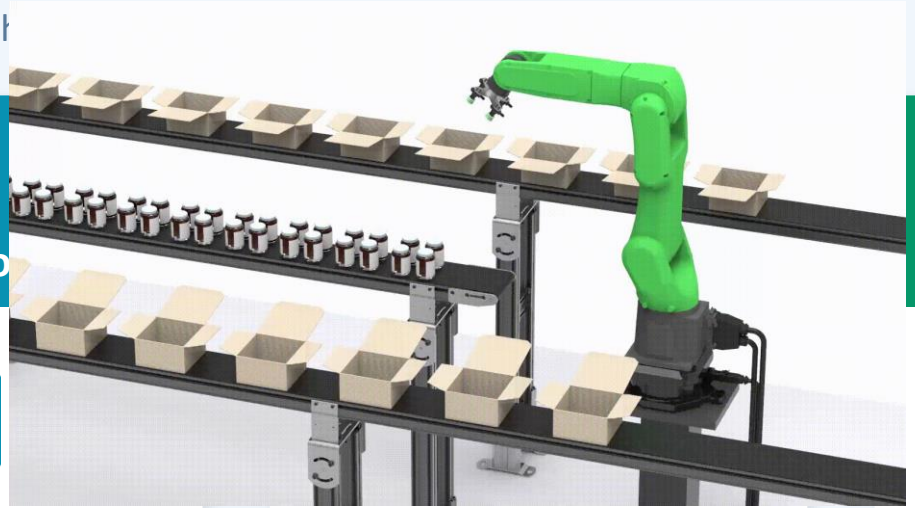
Decide what to do and find a safe path

3 Control

Send motor commands to execute the action

The Robotics Pipeline

Every robot needs th



1 Perceptio

Understand the world from sensors & cameras

3 Control

Send motor commands to execute the action

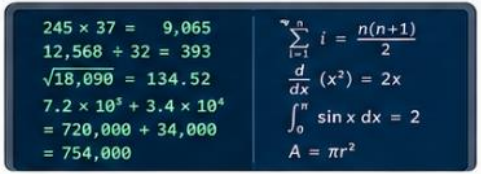
Decide what to do and find a safe path

Example: Robot picks up a cup → sees cup (Perception) → plans arm path (Planning) → moves motors (Control)

Easy for computers, hard for humans

1. 

Chess strategy

2. 

Fast calculation

3. 

Searching large data

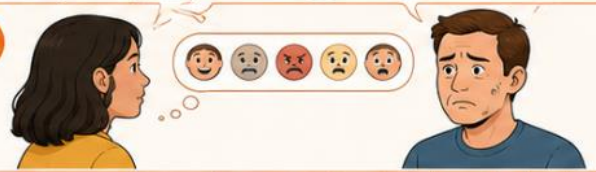
4. 

Pattern matching at scale

Easy for humans, hard for computers/robots

1. 
Walking in a messy room

2. 
Grasping irregular objects

3. 
Reading emotions

4. 
Everyday physical coordination

Physical intelligence is incredibly rich — balance, touch, vision, timing, all at once!

Moravec's Paradox

Easy for Computers

- Chess & strategy games
- Massive calculations
- Solving math problems
- Searching databases

vs

Hard for Robots

- Walking across a room
- Picking up objects
- Folding a shirt
- Opening a door

Physical intelligence is incredibly rich — balance, touch, vision, timing, all at once!

🎯 Exercise 1 — (5 minutes)

For each task: Is it easy for a human? Easy for a computer? Hard for a robot?

1. Chess

Easy for human? Hard for robot?

2. Walking in a messy room

Easy for human? Hard for robot?

3. Multiply 8273 × 4591

Easy for human? Hard for robot?

4. Pick up a soft toy

Easy for human? Hard for robot?

5. Search 1 billion records

Easy for human? Hard for robot?

6. Recognize a friend's mood

Easy for human? Hard for robot?

7. Drive in snow

Easy for human? Hard for robot?

8. Sort 1 million numbers

Easy for human? Hard for robot?

Why Learning-Based Robotics?

❌ Old Way: Hand-code Everything

- Write rules for every situation
- Breaks when environment changes
- Months of engineering per task
- Fails on messy real-world data

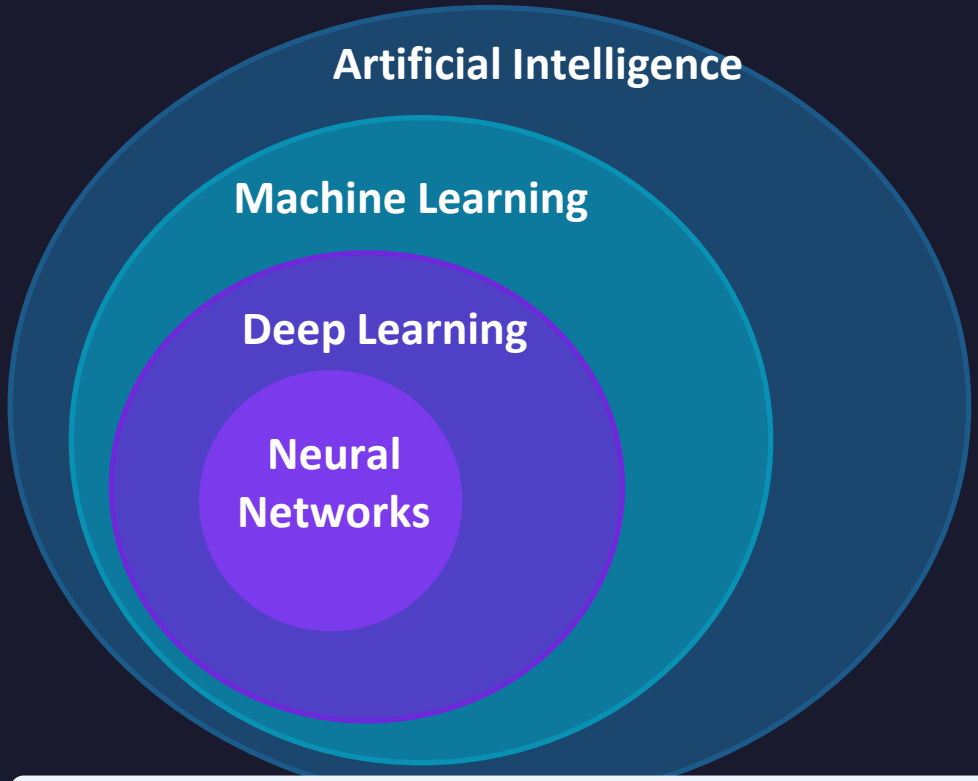
✅ New Way: Learn from Data

- Train on examples or experience
- Adapts to new situations
- One framework — many tasks
- Handles noise & variation

"Instead of programming every rule, we train a flexible model using data and optimization."

AI ⊃ ML ⊃ DL ⊃ NN

Every neural network is deep learning — but not every AI is a neural network



AI only (not ML)

- **Chess engine**
- **GPS routing**
- **Scripted chatbot** if 'hello' → say 'Hi!' — no training, just rules

ML (not Deep Learning)

- **Spam filter** Naive Bayes: word counts, one layer, no backprop
- **House prices** Linear regression: $\hat{y} = w \cdot x + b$, closed-form solution

Deep Learning (many NN architectures)

- **Image classifier** CNN: convolutional layers find edges and shapes
- **Speech recognition**

💡 Key insight: the circles nest — every DL model is ML, every ML model is AI — but not vice versa!

Supervised Learning

Learn from labeled input-output examples

Input
(x)

Model
(fθ)

Prediction
(ŷ)

True Label
(y)

Loss
L(θ)

Robotics Example

Input = camera image → Model = neural network → Output = which action a human would take

Training Set

Used to fit the model parameters

Test Set

Used to check generalization to new data

Google Colab — Part 1 (45 minutes)

Open the shared notebook → robotics_colab.ipynb

1 Open Google Colab

colab.research.google.com → open shared link

2 Run a cell

Click the cell, then press Shift + Enter

3 Say hello

Run Section 1.1 — print("Hello, Robotics!")

4 Move the robot

Section 1.2 — change start position and step size

5 Build an agent

Section 1.3 — agent that walks to a goal

6 Sensor noise

Section 1.4 — noisy sensor, try more readings

20-Minute Break

Be back at: _____

Coming up: Vectors · Loss function · Gradient descent · More Colab!

Vectors: A Robot's State

A vector is just a list of numbers — the robot's position, speed, sensor readings …

Robot in 2D space

$p = [\, x,\ \ y \,] = [\, 2,\ \ 5 \,]$

$\Delta p = [\, 3,\ -1 \,]$ (movement)

p_new = 

🎯 Quick Exercise

Robot starts at (1, 2)

Move 1: (+4, +3) → ?

Move 2: (−2, −1) → ?

Move 3: (+1, +5) → ?

Distance to Goal (Euclidean Norm)

$$\|p\| = \sqrt{(x^2 + y^2)}$$

In Colab:
`np.linalg.norm(goal - robot)`

→ Sections 2.1 & 2.2 in notebook

Example: difference = [3, 4] → $\sqrt{(9 + 16)} = \sqrt{25} = 5$

Vectors: A Robot's State

A vector is just a list of numbers — the robot's position, speed, sensor readings …

Robot in 2D space

$p = [\, x,\ y \,] = [\, 2,\ 5 \,]$

$\Delta p = [\, 3, -1 \,]$ (movement)

$p_{new} = p + \Delta p = [\, 5,\ 4 \,]$

🎯 Quick Exercise

Robot starts at (1, 2)

Move 1: (+4, +3) → ?

Move 2: (−2, −1) → ?

Move 3: (+1, +5) → ?

Distance to Goal (Euclidean Norm)

$$\|p\| = \sqrt{(x^2 + y^2)}$$

In Colab:
`np.linalg.norm(goal - robot)`

→ *Sections 2.1 & 2.2 in notebook*

Example: difference = [3, 4] → $\sqrt{(9 + 16)} = \sqrt{25} = 5$

Model · Loss · Gradient Descent

The Linear Model

$$\hat{y} = w \cdot x + b$$

w = weight (slope) **b** = bias (intercept)

The Loss Function

$$L = \text{mean}((\hat{y} - y)^2)$$

Measures how wrong predictions are.
Small loss = good model.

Gradient Descent — The Update Rule

$$\theta \leftarrow \theta - \alpha \cdot \nabla L(\theta)$$

θ = model parameters (w and b) **α** = learning rate — how big a step **∇L** = gradient — the uphill direction

⛰ Hill analogy: standing on a foggy hillside — you feel the slope — you step downhill — repeat until you reach the valley!

Google Colab — Part 2 (45 minutes)

2.3 **Build a linear model**
Define $\hat{y} = w \cdot x + b$ as a Python function. Change w and b.

2.4 **Plot noisy data**
Generate y = 2x + 1 + noise. See why the data is scattered.

2.5 **Compute the loss**
Try models with different w/b. Which has the smallest loss?

2.6 🎮 **Fit by hand!**
Change w and b until loss < 3. Race against your neighbour!

2.7 **Gradient descent**
Watch the loss fall automatically. Try α = 0.001, 0.01, 0.1.

Open Questions in Robotics

These are your generation's problems to solve

- How can robots learn from fewer examples?
- How can robots adapt to new tasks and environments?
- How do we keep robots safe and reliable?
- How can robots generalize beyond training data?
- How can robots learn when real-world data is expensive?
- Can we understand what a learned policy is actually doing?

Robotics + ML is one of the most exciting research frontiers of our time.

What You Learned Today

- Robots need Perception, Planning, and Control
- Moravec's Paradox: physical tasks are surprisingly hard for robots
- AI ⊃ ML ⊃ Deep Learning ⊃ Neural Networks (nested circles!)
- A vector is a list of numbers — it can represent robot state
- A linear model: $\hat{y} = w \cdot x + b$ · Loss: $L = \text{mean}((\hat{y} - y)^2)$
- Gradient descent: $\theta \leftarrow \theta - \alpha \cdot \nabla L(\theta)$ — always step downhill
- In Colab: built agents, added sensor noise, fitted a line by hand